Control.Concurrent.QSem: https://hackage.haskell.org/package/base/docs/Control-Concurrent-QSem.html

```
data QSem
newQSem :: Int -> IO QSem
waitQSem :: QSem -> IO ()
signalQSem :: QSem -> IO ()
```

QSem
Semaphore i.e. a lock

Example: basic usage

```
import Control.Concurrent.QSem
import Control.Exception (bracket, bracket_)
import Control.Concurrent (forkIO, threadDelay)

wait :: Int
wait = 2 * 1000 * 1000

createLock :: IO QSem
createLock = bracket
  (newQSem 0)                          -- create lock and notify that
it's free
  signalQSem
  return

runAction :: QSem -> IO a -> IO a
runAction sem = bracket_
  (waitQSem sem)                       -- wait until lock is free
  (signalQSem sem)                     -- free lock when done

startTest :: IO QSem
startTest = createLock >>= someAction "Start"

someAction :: String -> QSem -> IO QSem
someAction msg sem = do
  _ <- forkIO $ runAction sem $ threadDelay wait >> putStrLn msg
  return sem
```

e.g.

```
ghci> startTest >>= someAction "a"  >>= someAction "b"
ghci>
"Start"
... 2 seconds ...
"a"
... 2 seconds ...
"b"
... 2 seconds ...
```